BombLab 实验报告

第 1 关 函数调用与返回值

phase_01 先要求一个整数，再调用 p1_compute，将 eax 与栈上输入比较。p1_compute 依次调用 step2、step0、step1；跟踪参数传递后可见调用顺序与函数名后缀不同。r8d 保存 challenge 低 32 位，ecx 暂存 step2 返回值。

```
40206d: call p1_step2
40207a: call p1_step0
402083: call p1_step1
4020e0: cmp %eax,-0x14(%rbp)
```

恢复的计算关系

```
L = U(C); H = U(C >> 32)
a = U(L + 0x565f) XOR H
b = U(3 * L) XOR U(U(C >> 16) + 0x65f8)
answer = U(8 * b + (a XOR 0x5ea9))
```

这里 lea 完成乘法或加法，xor 改变位模式。必须保留 32 位截断，否则较大的 challenge 会使结果偏离程序检查值。

```
./defuser --seed 43 --phase 1 | ./bomb --seed 43 --phase 1
python3 public_test.py --phase 1
```




```
root@ics:~/home/sunnyxianbai/ics-2026-bomblab# python3 public_test.py --phase 1
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 4 第 1 关 GDB 比较 非公开 seed 43 通关与公开测试

seed 43 的 challenge 为 0x9cde98852e60034b，口令为 1340837198。GDB 在 0x4020e0 观察到 expected 与 actual 同为 1340837198；公开测试 10/10。

第 2 关 数组与递推循环

valid_u32_fields 的第二个参数为 6；sscanf 将六个整数存到 rbp−0x40 开始的连续数组。程序先检查首项，再以 rdx 为下标逐项比较。循环中 esi 保存上一项，移位量 ecx 从 15 起每轮增加 2。

```
402160: shr $0x2c,%rax       # C >> 44
402180: lea (%rsi,%rsi,4),%esi # 5 * previous
402183: shr %cl,%rax
402186: and $0x3f,%eax
402189: xor %edi,%eax
402191: cmp %eax,%esi
```

恢复的计算关系

```
v[0] = (((C >> 44) * 2) & 255) + 11
for i = 1..5:
   t = (C >> (13 + 2*i)) & 63
   v[i] = U(5 * v[i-1] + (t XOR (i+5)))
```

数组共六项，不是执行六次递推。条件断点使用 break *0x402191 if $rdx == 5，以观察最后一项比较。

```
./defuser --seed 43 --phase 2 | ./bomb --seed 43 --phase 2
python3 public_test.py --phase 2
```

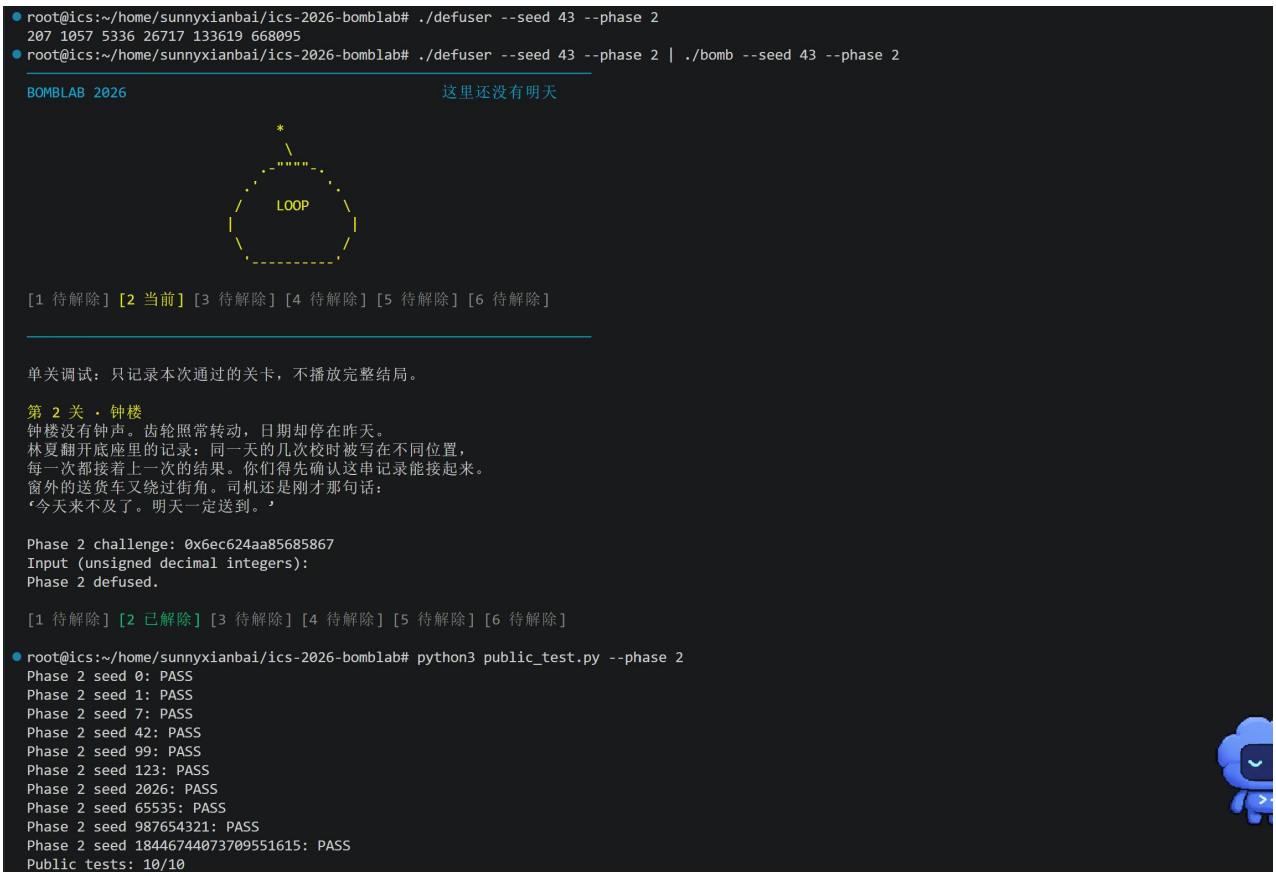


图 5 第 2 关末项比较 非公开 seed 验证与公开测试

seed 43 的 challenge 为 0x6ec624aa85685867，六项为 207 1057 5336 26717 133619 668095。最后一次比较的两侧同为 668095，公开测试 10/10。

第 3 关 switch 与跳转表

输入包含选择号和计算结果。第一项必须等于 C & 7；第二项由跳转表确定。0x405820 开始的八个地址指向跳转桩，再跳到 case 函数，因此不能直接把选择号当作 case 函数后缀。令 x = U(C >> 3) XOR U(C >> 32)。

402301: and $0x7,%edi
40225d: notrack jmpq *0x405820(,%rax,8)
40231e: cmp %eax,-0x14(%rbp)

选择号	实际 case	第二项的公式
0	1	U(2*x) XOR 0x39f1
1	6	U((x XOR 0xdab3) + (x >> 2))
2	2	U((x XOR 0xc35f) + (x >> 2))
3	3	U(5*x + 0x5ad5)
4	5	U(2*x) XOR 0x0be1
5	0	U(5*x + 0x7d78)
6	7	U(5*x + 0x3a7f5)
7	4	U(3*x + 0x2ec4)

./defuser --seed 43 --phase 3 | ./bomb --seed 43 --phase 3
python3 public_test.py --phase 3

```
root@ics:~/home/sunnyxianbai/ics-2026-bomblab# ./defuser --seed 43 --phase 3
7 2577469569
root@ics:~/home/sunnyxianbai/ics-2026-bomblab# ./defuser --seed 43 --phase 3 | ./bomb --seed 43 --phase 3
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]

root@ics:~/home/sunnyxianbai/ics-2026-bomblab# python3 public_test.py --phase 3
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

图 6 第 3 关选择号 7 对应 case 4 以及运行验证

seed 43 的 challenge 为 0xd49ffe504bfdab7f，口令为 7 2577469569。GDB 第二项比较相等，公开测试 10/10；直接 challenge 检查另外覆盖了全部八个选择号。

第 4 关 递归路径与返回组合

phase_04 从 challenge 生成 target 与 key，然后调用 p4_walk(target, 0, 127, key)。函数用区间中点划分左右子区间；找到 target 后给出基本值，返回时逐层乘 3、异或中点与 key 的和，再异或分支常量。

402338: sub %esi,%edi; shr %edi; add %esi,%edi
402350: jb 402370 # target < mid
40235c: lea (%rax,%rax,2),%eax # 3 * recursive_result
402366: xor $0xb,%eax # right branch
402384: xor $0x1d,%eax # left branch

target = C & 127; key = U(C >> 7) XOR U(C >> 32)
mid = lo + ((hi-lo) >> 1)
F(mid,lo,hi,key) = U(mid+0x63) XOR key
target < mid: U(3*F(target,lo,mid-1,key)) XOR U(mid+key) XOR 0x1d
target > mid: U(3*F(target,mid+1,hi,key)) XOR U(mid+key) XOR 0x0b

./defuser --seed 43 --phase 4 | ./bomb --seed 43 --phase 4
python3 public_test.py --phase 4

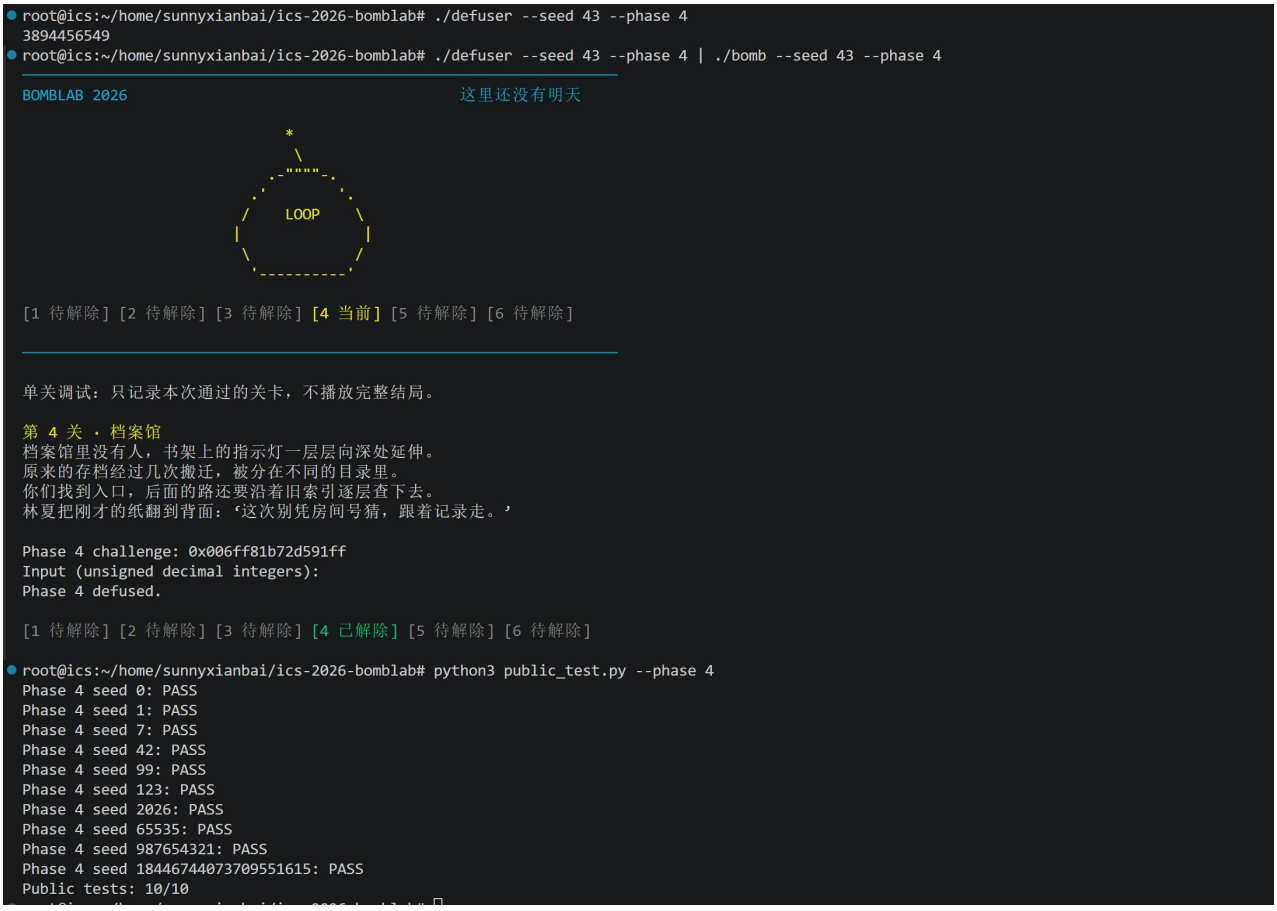


图 7 第 4 关 target 127 的递归区间与最终比较

seed 43 的 challenge 为 0x006ff81b72d591ff，target=127，key=915034936。GDB 记录区间逐步收缩至 [127,127]，最终口令为 3894456549，公开测试 10/10。直接 challenge 0..127 覆盖全部 target。

第 5 关 结构体与链表遍历

p5_traverse 从头指针表选择起点，通过偏移 8 的 next 字段遍历，并读出偏移 0 的 value 与偏移 4 的 id。第二个函数参数是保存末节点 id 的指针。输入第一项为累计值，第二项为末节点 id，不能把节点索引当作 id。

```
402434: mov 0x405b60(,%rax,8),%rdx  # head pointer
402449: mov 0x4(%rdx),%esi          # id
402456: add (%rdx),%eax             # value
402467: mov 0x8(%rdx),%rdx          # next
402478: mov %esi,(%r8)              # last id
```

```
node = heads[C & 7]; hops = (C >> 3) & 3
acc = U(C >> 6) XOR U(C >> 32) XOR 0x857f
repeat hops+1 times:
   acc = U(3*acc + node.value) XOR node.id
   last_id = node.id; node = node.next
answer = acc, last_id
```

首节点在循环之前就已参与原程序的计算，所以总处理数量为 hops+1，即 1 到 4 个节点。代码使用前述只读表对应的索引，并对索引越界返回失败。

```
./defuser --seed 43 --phase 5 | ./bomb --seed 43 --phase 5
python3 public_test.py --phase 5
```

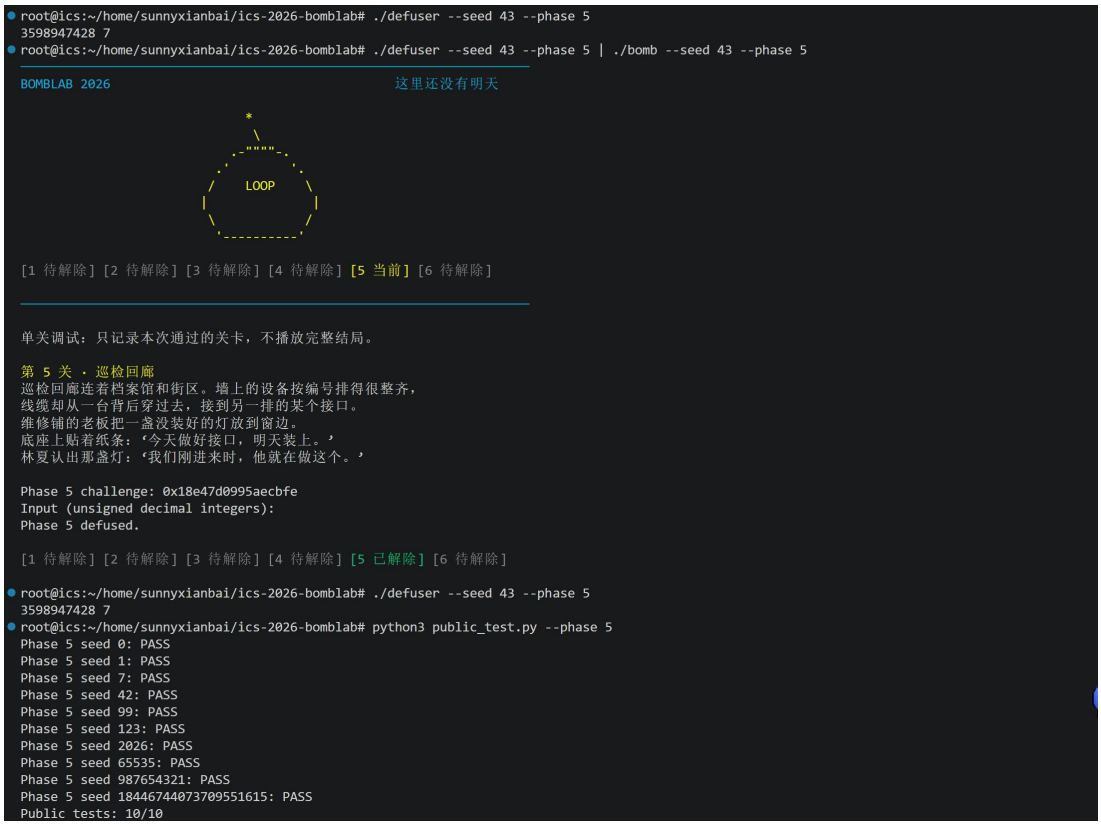


图 8 第 5 关累计值 末节点 id 及运行验证

seed 43 的 challenge 为 0x18e47d0995aecbfe，起点索引 0，路径为 0→11→10→3，共四个节点；末节点 id 为 7。口令为 3598947428 7，公开测试 10/10。

第 6 关 函数指针与间接调用

p6_dispatch 从同一指针表中执行两次间接调用。第一下标为 C & 7；第二次的字节偏移为 C & 0x38，除以每项 8 字节后等于 (C >> 3) & 7。表中存在重复函数，必须保留实际映射。

```
402596: callq *0x405e00(,%rax,8)
40259d: xor $0x6aed,%eax
4025a4: callq *0x405e00(%rbx)
4025af: add $0xe0de,%eax
```

表索引	op	运算 g(x)
0 1	0	U((x XOR 0x23c1) + 4*x)
3 4	1	ROR32(x,2)
6	2	U((x << 7) + 0xe07c) XOR U(3*x)
5	3	ROL32(x,1) XOR 0x91d8
2	4	U((x >> 2 XOR x) + 0xd265)
7	5	U(x + 0x809f) XOR (x >> 2)

```
    x = U(C >> 6) XOR U(C >> 32)
a = table[C & 7](x)
answer = U(table[(C >> 3) & 7](a XOR 0x6aed) + 0xe0de)
```

```
./defuser --seed 43 --phase 6 | ./bomb --seed 43 --phase 6
python3 public_test.py --phase 6
```



图 9 第 6 关两次间接调用的计算结果验证

seed 43 的 challenge 为 0x523fb4dd5d162e88，两个表下标分别为 0 和 1，都指向 op0；口令为 3597424648。公开测试 10/10，直接 challenge 0..63 覆盖全部 64 种下标组合。

Honor 入口布局与三个校验值

Honor 需要不带 --phase 的完整运行，且从文件或管道提供第七行。驱动先用前六关答案计算 context，再把 Honor challenge 与 context 传给 solve_secret。程序的 secret_gate 将输入复制到局部对象，并在相邻字段检查一个标记。

```
40273b: lea -0xd0(%rbp),%r12
402761: sub $0x24,%rax; cmp $0x7f,%rax
402786: callq secret_copy
40278b: cmpl $0x4e45504f,-0xb0(%rbp)
40279d: lea -0xac(%rbp),%rdi
4027a4: callq secret_phase
```

相对起点的字节偏移	填入内容	作用
0..31	32 个 ASCII 字符 A	填满前部缓冲区
32..35	OPEN	小端序对应 0x4e45504f
36 起	三个十进制整数及结尾 NUL	由 secret_phase 解析并校验

长度检查要求输入字符串长 36 到 163 字节。本解法第七行最长 59 字节，含结尾 NUL 最多 60 字节，落在局部对象范围内；不覆盖返回地址，不使用 shellcode，也不通过调试器修改内存。

三个整数的关系

以下令 C 为 Honor challenge，T 为 context；所有中间计算按 U 保留 32 位。secret_mix 把移位异或与加法组合，再与 context 派生值混合。

```
    k = U(T) & 3
a = U(C) XOR U(C >> 32); b = U(T) XOR U(T >> 32)
x = a XOR b
mix = U(((x >> 1) XOR 0x95a3) + (x XOR 0x3199))
v = mix XOR (U(k*0x3199) XOR U(T >> 16))
w = v XOR U(b + k + 0x95a3 + (a >> 1))
payload = "A" repeated 32 times + "OPEN" + decimal(k,v,w)
```

第七行的数字从偏移 36 开始，不需要在 OPEN 与第一个数字之间添加分隔符；三个数字之间保留单个空格。context 由驱动计算，求解函数不自行读取前六行或重算文件内容。

关键校验指令

```
    4026a8: cmp %ecx,-0x2c(%rbp)   # k
4026dc: cmp %eax,-0x28(%rbp)   # v
4026ef: cmp -0x24(%rbp),%eax   # w
```

Honor 非公开 seed 验证

```
    { ./defuser --seed 43; ./defuser --secret --seed 43; } > evidence/secret-43.txt
gdb -q -batch -x evidence/secret.gdb ./bomb
{ ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
```

在 secret_gate 入口按调用约定观察 rsi、rdx；在复制之后检查标记，在两个比较点核对期望值与输入。

观察项目	seed 43 实际记录
Honor challenge	0xe22a53d064a0b63a
context	0x425ecdadeff5e805
marker	0x4e45504f
k v w	1 2367666481 2112788336
通过标志	Secret phase defused.

```
root@ics:~/home/sunnyxianbai/ics-2026-bomblab# { ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
```

BOMBLAB 2026　　这里还没有明天



[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

一个普通的晚上，你和林夏在实验室接上了一台旧设备，
却误入了一座陌生的数字城市。住在这里的，是 AI。
街边的维修铺还开着，老板正低头摆弄一盏灯。
再往前走，维护站的门半开着，里面只有一台旧终端。
屏幕上留着一条消息：

> 没想到这里还会有人来。
> 祝你们好运。
> — sky

窗口角落留着‘返回实验室’的按钮。
林夏把外套搭在椅背上：‘先看看？’
维护面板上画着一枚炸弹，旁边亮着六项未完成的交接检查。
它不是街上的爆炸物，只是旧系统给这次维护选的图标。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

```
Phase 1 challenge: 0x9cde98852e60034b
Input (unsigned decimal integers):
Phase 1 defused.
```

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

```
Phase 2 challenge: 0x6ec624aa85685867
Input (unsigned decimal integers):
Phase 2 defused.
```

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

```
Phase 3 challenge: 0xd49ffe504bfdab7f
Input (unsigned decimal integers):
Phase 3 defused.
```

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’



```
Phase 4 challenge: 0x006ff81b72d591ff
Input (unsigned decimal integers):
Phase 4 defused.
```

```
最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x18e47d0995aecbfe
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x523fb4dd5d162e88
Input (unsigned decimal integers):
Phase 6 defused.

终端逐项读过记录，最后停在‘保存完成’。
林夏没有马上起身。你们等了一会儿，屏幕上的日期向前走了一天。
这次没有恢复旧存档。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。
你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。

阅读器亮起来。最先出现的不是代码，是两份很早的留言。

  许遥：

  北方那边的通知到了，我准备去。
  那个项目我想试很久了。你帮我看申请的时候，我还说不一定选得上，
  现在真能去了，反而有点不习惯。以后遇到问题，
  不能再把椅子滑到你旁边，问一句‘你看这是不是有毛病’。

  你想留下继续手里的事，我记得你为它准备了多久。
  我也想看看，最后能做成什么样。
  这里没做完的几处，我把记录整理好了。
  你有空就改，没空先忙自己的事。

  走之前出去吃顿饭吧。这次别在实验室凑合了。

  —— sky

隔了几天，许遥在后面添了一份回复。

  吃饭的时间你定。
```



相同 seed 下六关和 Honor 全部通过。公开 Honor 测试另有 10/10 通过记录，见下一节；256 个非公开 seed 也进行了完整联合运行。

全部公开测试

```
    make test
python3 evidence/verify_extended.py
```

make test 使用模板公开测试，六关通过后继续检查 Honor。扩展脚本使用固定随机源，排除十个公开 seed 后选取 256 个 seed，其中包含 43 和若干整数边界。每个 seed 都运行六行正式答案与 Honor 第七行，再用原始 bomb 校验。

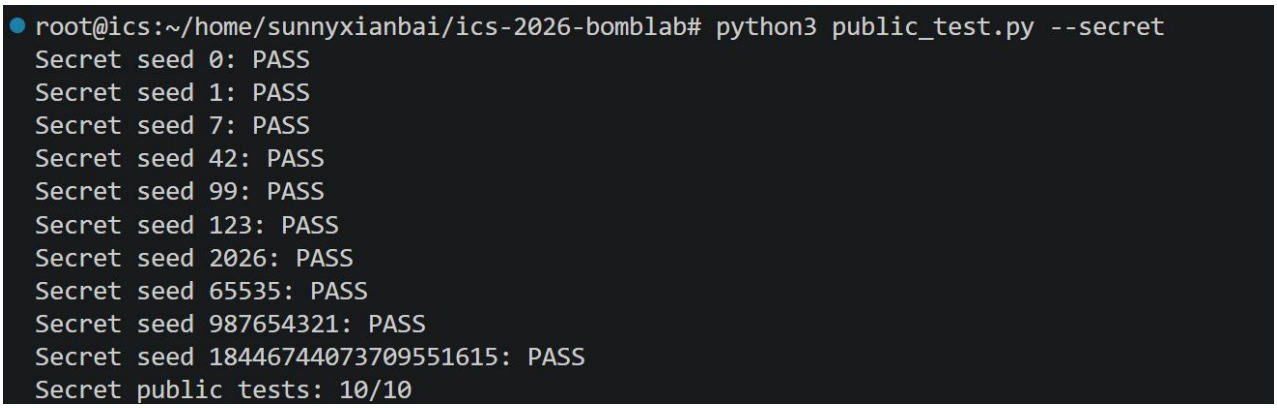


图 11 公开测试汇总 六关 60/60 与 Honor 10/10